OS Lab Assignment 9: Page Tables, Page Faults and Page Replacement

Built on a clean copy of xv6-public (x86). The Makefile drops `-Werror` and uses `-smp N,sockets=N`. New system calls: `vmprint` (22) and `getfaults` (23). Every kernel change is tagged `// A9 Qn`. Run with `make qemu-nox CPUS=1` and then `pttest`, `faulttest` and `pagerepl`. Logs are in `output_logs/` and screenshots in `screenshots/` (appended below).

Q1: Printing the page table (`vmprint`)

Changes. `vmprint(pgdir)` in `vm.c` follows `freevm()`. The outer loop runs over the PDEs from 0 to `PDX(KERNBASE)`, so the kernel half is skipped. Each present PDE counts as one page-table page. The inner loop walks that table (`P2V(PTE_ADDR(pde))`) and prints every present PTE as `PGADDR(i,j,0)`, `PTE_ADDR(pte)` and the P/W/U flags. The totals count pages that have `PTE_U` set, and the page-table count includes the page directory (+1). The syscall is wired through `syscall.h/.c`, `sysproc.c`, `user.h`, `usys.S` and `defs.h`.

Output. At start-up there are 3 PTEs: code/data `0x0` (PWU), the guard page `0x1000` (`PW-`, U cleared by `exec`), and the stack `0x2000`. That gives **2 user pages and 2 page-table pages**. After `sbrk(4*4096)`, `pttest` writes to the 4 pages (needed because of Q2's lazy allocation), and exactly **4 new PWU pages** appear at `0x3000`–`0x6000`, giving **6 user pages**. Their physical frames are not contiguous. The page-table count stays at 2 because everything is still inside the first 4 MB.

Q2: Lazy allocation and page faults

Changes.

- `sys_sbrk`: for `n > 0` it only does `sz += n`, after rejecting overflow and `>= KERNBASE`. `n < 0` still goes through `growproc`.
- `copyuvm`: `continue`s on a missing page table or a non-present PTE instead of panicking.
- `deallocuvm`: already skips non-present PTEs (commented).
- `mappages`: made non-static.
- `trap.c`: `case T_PGFLT` calls `lazyfault()`. It reads `rcr2()` and rejects protection faults (`err & 1`, e.g. the guard page) and addresses `>= sz`. Otherwise it does `kalloc()`, `memset(0)`, `mappages(PGROUNDDOWN(va), PTE_W|PTE_U)` and `p->faults++`, and returns so the instruction restarts. On an invalid fault in user mode it prints the pid and address and sets `killed`. A kernel-mode fault inside a system call (e.g. `read()` into an untouched buffer) is served the same way; only a kernel fault on a bad address still panics.
- `proc.h` / `proc.c`: new `int faults`, zeroed in `allocproc`.
- `exec.c`: resets `faults` when the new image is committed.
- `getfaults()` returns `myproc()->faults`.

Output. After `sbrk(32*4096)` the count is still **0**. Each first write to pages 0, 7, 15, 23 and 31 adds one fault, giving **5**. Rewriting pages that are already mapped adds none. A forked child that writes to `sz + 4096` is killed (`pid 4 ... invalid page fault at addr 0x24000 (sz 0x23000)`), and the parent continues. Stock `usertests` still prints **ALL TESTS PASSED**.

Problem fixed. The count first came out as 7. The shell's forked child mallocs during `parsecmd()` (2 lazy faults) before `exec`, and the counter carried over. Resetting it in `exec()` fixed this.

Q3: FIFO vs LRU (`pagerepl`)

Program. `pagerepl [frames refs...]` (refs may be comma-separated, because xv6's shell allows only 10 arguments). With no arguments it uses the given string and 3 frames. `simulate()` keeps `frame[]` and `last[]`. FIFO evicts the frame at a rotating pointer (the oldest load). LRU updates `last[]` on every hit and evicts the minimum (empty frames have -1, so they fill first). The program prints the frames and hit/FAULT for each reference, then a table for 3, 4 and 5 frames. Hit ratios are printed with integer math, since xv6's `printf` has no `%f`.

Frames	FIFO faults	FIFO hit ratio	LRU faults	LRU hit ratio
3	**15**	25.00%	**12**	40.00%
4	10	50.00%	8	60.00%
5	9	55.00%	7	65.00%

LRU wins because it keeps the frequently reused page 0, which FIFO evicts just for being old. Six faults in every row are compulsory (6 distinct pages). With `1,2,3,4,1,2,5,1,2,3,4,5`, FIFO shows **Belady's anomaly** (9 faults with 3 frames, 10 with 4). LRU is a stack algorithm, so it never does (10, then 8).

Appendix: terminal screenshots

Q1: pttest

```
$ make qemu-nox CPUS=1
...
$ pttest
=== page table at start-up ===
VA          PA          FLAGS
0x0  0xdee8000  PWU
0x1000  0xdee6000  PW-
0x2000  0xdee5000  PWU
valid user pages: 2, page-table pages: 2

=== after sbrk(4 * 4096), new region 0x3000 - 0x7000 ===
VA          PA          FLAGS
0x0  0xdee8000  PWU
0x1000  0xdee6000  PW-
0x2000  0xdee5000  PWU
0x3000  0xdfbc000  PWU
0x4000  0xdf76000  PWU
0x5000  0xdfbf000  PWU
0x6000  0xdfc0000  PWU
valid user pages: 6, page-table pages: 2
```

Q2: faulttest

```
$ make qemu-nox CPUS=1
...
$ faulttest
faults at start: 0
sbrk(32 * 4096) -> 0x3000, faults after sbrk: 0
wrote page 0 (addr 0x3000), faults = 1
wrote page 7 (addr 0xA000), faults = 2
wrote page 15 (addr 0x12000), faults = 3
wrote page 23 (addr 0x1A000), faults = 4
wrote page 31 (addr 0x22000), faults = 5
rewrote 2 already-touched pages, faults = 5
page faults handled: 5 (expected 5)
child 4: writing above sz at 0x24000
pid 4 faulttest: invalid page fault at addr 0x24000 (sz 0x23000)--kill proc
parent 3: child was killed, parent still alive, faults = 5
```

Q2: usertests still pass

```
$ make qemu-nox CPUS=1
...
$ usertests
usertests starting
arg test passed
createdelete test
createdelete ok
linkunlink test
...
fork test OK
bigdir test
bigdir ok
uio test
pid 593 usertests: trap 13 err 0 on cpu 0 eip 0x35e3 addr 0x24d1d004--kill proc
uio test done
exec test
ALL TESTS PASSED
```

Q3: pagerepl (FIFO/LRU trace and table)

```
$ make qemu-nox CPUS=1
...
$ pagerepl
reference string (20 refs): 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

--- FIFO, 3 frames ---
ref 7 :  7  -  -   FAULT
ref 0 :  7  0  -   FAULT
ref 1 :  7  0  1   FAULT
ref 2 :  2  0  1   FAULT
ref 0 :  2  0  1   hit
ref 3 :  2  3  1   FAULT
ref 0 :  2  3  0   FAULT
ref 4 :  4  3  0   FAULT
ref 2 :  4  2  0   FAULT
ref 3 :  4  2  3   FAULT
ref 0 :  0  2  3   FAULT
ref 3 :  0  2  3   hit
ref 2 :  0  2  3   hit
ref 1 :  0  1  3   FAULT
ref 2 :  0  1  2   FAULT
ref 0 :  0  1  2   hit
ref 1 :  0  1  2   hit
ref 7 :  7  1  2   FAULT
ref 0 :  7  0  2   FAULT
ref 1 :  7  0  1   FAULT
FIFO total page faults: 15

--- LRU, 3 frames ---
ref 7 :  7  -  -   FAULT
ref 0 :  7  0  -   FAULT
ref 1 :  7  0  1   FAULT
ref 2 :  2  0  1   FAULT
ref 0 :  2  0  1   hit
ref 3 :  2  0  3   FAULT
ref 0 :  2  0  3   hit
ref 4 :  4  0  3   FAULT
ref 2 :  4  0  2   FAULT
ref 3 :  4  3  2   FAULT
ref 0 :  0  3  2   FAULT
ref 3 :  0  3  2   hit
ref 2 :  0  3  2   hit
ref 1 :  1  3  2   FAULT
ref 2 :  1  3  2   hit
ref 0 :  1  0  2   FAULT
ref 1 :  1  0  2   hit
ref 7 :  1  0  7   FAULT
ref 0 :  1  0  7   hit
ref 1 :  1  0  7   hit
LRU total page faults: 12

frames | FIFO faults | FIFO hit ratio | LRU faults | LRU hit ratio
-------+-------------+----------------+------------+--------------
   3   |     15      |     25.00%     |     12     |    40.00%
   4   |     10      |     50.00%     |      8     |    60.00%
   5   |      9      |     55.00%     |      7     |    65.00%
```

Q3: Belady's anomaly

```
$ make qemu-nox CPUS=1
...
$ pagerepl 3 1,2,3,4,1,2,5,1,2,3,4,5
reference string (12 refs): 1 2 3 4 1 2 5 1 2 3 4 5

--- FIFO, 3 frames ---
ref 1 :  1  -  -   FAULT
ref 2 :  1  2  -   FAULT
ref 3 :  1  2  3   FAULT
ref 4 :  4  2  3   FAULT
ref 1 :  4  1  3   FAULT
ref 2 :  4  1  2   FAULT
ref 5 :  5  1  2   FAULT
ref 1 :  5  1  2   hit
ref 2 :  5  1  2   hit
ref 3 :  5  3  2   FAULT
ref 4 :  5  3  4   FAULT
ref 5 :  5  3  4   hit
FIFO total page faults: 9

--- LRU, 3 frames ---
ref 1 :  1  -  -   FAULT
ref 2 :  1  2  -   FAULT
ref 3 :  1  2  3   FAULT
ref 4 :  4  2  3   FAULT
ref 1 :  4  1  3   FAULT
ref 2 :  4  1  2   FAULT
ref 5 :  5  1  2   FAULT
ref 1 :  5  1  2   hit
ref 2 :  5  1  2   hit
ref 3 :  3  1  2   FAULT
ref 4 :  3  4  2   FAULT
ref 5 :  3  4  5   FAULT
LRU total page faults: 10

frames | FIFO faults | FIFO hit ratio | LRU faults | LRU hit ratio
-------+-------------+----------------+------------+--------------
   3   |      9      |     25.00%     |     10     |    16.66%
   4   |     10      |     16.66%     |      8     |    33.33%
   5   |      5      |     58.33%     |      5     |    58.33%
```